

This assignment focuses on methods, top-down design, and building a complete turn-based combat system. You will demonstrate your ability to plan a system architecture, implement methods, and create a working program using modular design principles.

Submission Requirements:

- Read the [Organizational Guidelines](#) on Moodle, and adhere to them!
- Submit a PDF document containing your **solution idea, code, and tests for each exercise**
- Include all Java source files (.java) in a ZIP archive
- **Testing:** Writing tests is OPTIONAL. However, for BOTH exercises combined, you must:
 - Play through your own game at least once and show the complete output
 - Include at least one example of invalid input (wrong player action) being handled correctly
- Use the naming convention: MC_UE05_LASTNAME.pdf and MC_UE05_LASTNAME.zip

Important: This assignment emphasizes planning and working code. Your solution idea should clearly explain your approach, architecture, and method structure. Working code that uses methods is more important than perfect refactoring - if you struggle with method extraction, focus on getting a working solution first.

Exercise 1 (18 points) Turn-Based Combat System - Basic Features

Implement a complete turn-based combat system following the specification in the Combat System Guide (pdf on Moodle). Your program should be called `TurnBasedCombat.java`.

Note that **ONLY** the base features (1.1.2 Required Features) are needed for this exercise.

The solution idea can be in any format you prefer (text description, method map, flowchart, pseudocode, etc.) but must be clear and comprehensive.

Coding Approach:

This assignment emphasizes working code over perfect refactoring. If you struggle with method extraction, it's better to get something working than to produce perfect code that doesn't work. However, you **must** use methods to organize your code - avoid putting everything in `main()`.

Example Output Format (feel free to get creative):

```
1  =====
2      TURN-BASED COMBAT SYSTEM
3  =====
4  Welcome to the combat arena!
5
6  Game Initialized!
7  Player HP: 100/100
8  Enemy HP: 100/100
9  Starting Turn 1
10 -----
11
12 =====
13 Turn 1
14 =====
15 Player:  HP=100/100  Armor=0
16 Enemy:   HP=100/100  Armor=0
17
18 Choose your action:
19     1 = Attack
20     2 = Heal
21     3 = Defend
22 Enter choice (1-3): 1
23 Player attacks for 25 damage!
24 Enemy HP: 75/100
25
26 Enemy attacks for 25 damage!
27 Player HP: 75/100
28
```

```

29  -----
30  [... continues until game over ...]
31  =====
32          GAME OVER
33  =====
34  Player wins!
35  Final Turn: 8
36  Player Final HP: 45/100
37  Enemy Final HP: 0/100
38  =====

```

Exercise 2 (6 points) Advanced Features Extension

Extend your combat system with **at least one** advanced feature from the list below, OR implement a custom feature of your own design (with description).

Advanced Feature Options:

- **Critical Hits:** 20% chance for attacks to deal 1.5x damage
- **Enhanced Combat Feedback:** Improved console output formatting and combat log
- **Improved Enemy AI:** More sophisticated decision-making based on combat situation
- **Game Balance Constants:** Use static final constants for all game values
- **Status Effects System:** Poison (damage over time) or Regeneration (healing over time)
- **Combo System:** Bonus effects when same action is used consecutively
- **Special Moves with Cooldowns:** Powerful abilities with turn-based cooldowns
- **Battle Statistics Tracking:** Track damage dealt, critical hits, combos, etc.
- **Multiple Rounds / Best of Series:** Tournament mode with multiple rounds
- **Difficulty Levels:** Adjustable difficulty (Easy, Normal, Hard)
- **Berserker Mode:** Low HP mode with damage bonus and defense penalty
- **Counter-Attack System:** Defensive reactions that counter-attack
- **Turn Order Variations:** Speed/initiative system determining turn order
- **Action Prediction Bonus:** Predict enemy actions for strategic bonuses

Or Custom Feature:

If you implement a custom feature of your own design, provide a clear description of:

- What the feature does
- How it enhances gameplay
- How it's implemented in code

Submission Requirements:

- Description of chosen feature(s) and how they work
- Code implementation (may extend your Exercise 1 code or be separate)
- Demonstration/output showing the feature(s) working